Work Notes: 3 to 4 Nov 2026 (Functions, campaign flags, broker commission workbook)

Nov 4, 2026 · @tanvik

Overview

This page covers 3 and 4 Nov, everything after the 2 Nov notes. Two threads ran through it: client-level reports built from the ledger (deposits, a campaign flag, trading days, broker commission), and learning to package code as functions and modules instead of copying cells.

#	Task	Tool	Key idea to learn
1	Lambdas and power functions notes	pandas	Vectorised first; `apply` last
2	Deposits and withdrawals, net of cancellations	pandas	`.where()` columns, then one `groupby().sum()`
3	`np.where` with AND / OR	pandas	`&`, `|`, `~`, and brackets around each condition
4	Campaign flag	pandas	"First X": sort, keep first, then test
5	Trading days and their spread	pandas	`nunique` of dates; joining dates per client
6	Excel export in the MIS design	openpyxl	Style the sheet pandas just wrote
7	Broker summary, two sheets	pandas	Combined brokerage vs client commission
8	Importing your own modules	Python	`sys.path.insert(0, folder)`
9	Writing functions	Python	Three levels: wrap, add parameters, add checks

The end-of-day review comes first, then one section per task, then the problems log, cheat sheet and open items.

End-of-day review (day 5 of 7)

You asked about 19 questions over the two days: 9 pandas, 4 on importing modules, 3 on writing functions, 2 on Excel styling, and the lambda notes. The biggest shift is in the questions themselves: "what does the 0 mean", "describe the flow", "I'm a newbie, show me simple first". You're asking to understand the code, not only to get it working, and starting to package work as functions and modules. That's the core habit of analytics engineering.

Solved

- Deposits and net withdrawals per client in one `groupby`, cancellations subtracted.
- `Traded Zero`, `Campaign` (first deposit 1,000 on 25 Aug), `Trading Days`, and the trading dates per client and per month.
- A broker commission workbook with Brokers and Clients sheets in the MIS design.
- `ledger_loader` imported from another folder, and why `insert(0, ...)` needs the 0.
- The P&L chain written as a function at three levels, checked against the original `pnl`.

Not confirmed yet

- Whether the `balance` line was changed to sort by `CreatedAt`.
- Whether the NBP rows are back in the broker calculation (filter `led_oct`, not `led_oct_bill`).
- Which brokerage rule the payout follows: the broker's combined loss, or the sum of client commissions.

Mistakes that came back

Pattern	Earlier	This time
Sorting by the key, not the date, for "latest"	29 and 30 Oct	`balance` sorted by `ClientID` (3 Nov); still in the code pasted later
Filtering away rows needed later	Wrong folder (31 Oct)	`led_oct_bill` dropped the NBP rows before the NBP filter ran
Request not precise enough	30-day columns (2 Nov)	"1,000 deposit on 25 Aug" turned out to mean "**first** deposit"

Pattern	Earlier	This time
Brackets	—	`np.where((a) & (b, 'Yes', 'No'))`

The latest-row sort is now the most repeated mistake in the journal. A one-line habit fixes it: **whenever you write `drop_duplicates(keep='first')`, the line before must sort by a date.**

Gaps to study

- **Functions:** parameters, defaults, `return`, and testing against a known result.
- **Boolean logic in pandas:** `&`, `|`, `~`, brackets, `isin`, `between`.
- **Order of filters:** which rows each step keeps, checked with `value_counts()` after each filter.

Study programme check

No separate study chats since 29 Oct. The lambda notes and the three function levels are the study material from these two days, and the workbook is still waiting for its first `progress()`.

Focus for tomorrow

Write `trading_days(led, start, end)` at Level 2 and check it against your existing `Trading Days` column with `assert_frame_equal`.

1. Lambdas and power functions

The notes are in Lambdas and Power Functions. The core rule: a lambda that receives a **whole column or table** is fine; one that runs **once per row** is the slow last resort.

Measured on 500,000 rows

Slow	Fast	Faster by
`df.apply(lambda r: r.a + r.b, axis=1)`	`df.a + df.b`	934×
`s.apply(lambda x: x * 2)`	`s * 2`	182×

Slow	Fast	Faster by
`apply(if/elif function)`	`np.select`	13×
`agg(lambda x: ...df.loc[x.index]...)`	`.where(cond, 0)` then `sum`	6×

One surprise: on pandas 3, `.str.split('|').str[1]` was the **slowest** way to split text, 6 times slower than a plain lambda. `.str.replace` with a regex was the fastest.

Good places for a lambda

- `key=` in `sorted` / `max`: `max(files, key=lambda f: f.stat().st_mtime)`.
- `.assign(new=lambda d: ...)` and `.loc[lambda d: ...]` in method chains.
- `.rename(columns=lambda c: c.strip())`.
- Joining text per group: `agg(lambda s: ', '.join(s))`.

2. Deposits and withdrawals, net of cancellations

Build each amount as its own column with `.where()`, then sum once per client. The `pivot_table` with two lambdas that look rows up again became three short lines, and the cancellation subtraction is part of the column.

```
lt = req_led['LedgerType'].str.strip().str.upper()
cr = req_led['Credit'].fillna(0)
dr = req_led['Debit'].fillna(0)

one_pivot = (req_led
    .assign(Deposit=cr.where(lt.eq('DEPOSIT'), 0),
            Withdraw=dr.where(lt.eq('WITHDRAW'), 0) -
cr.where(lt.eq('WITHDRAW CANCELLED'), 0))
    .groupby('ClientID', as_index=False)[['Deposit', 'Withdraw']]
    .sum())
```

How it works

- **`cr.where(lt.eq('DEPOSIT'), 0)`** keeps the Credit on deposit rows and puts 0 on every other row.
- **Each row adds `+debit`, `-credit` or `0` to `Withdraw`**, so the per-client sum is withdrawals minus cancellations.

- **`.fillna(0)` matters for the subtraction:** a blank Credit would make that row's result blank, and `sum` would silently skip it.
- **`.str.strip().str.upper()`** makes `'Withdraw Cancelled '` match `'WITHDRAW CANCELLED'`.

The alternative: pivot by type, then calculate

`pivot_table(columns='LedgerType', values=['Credit', 'Debit'], aggfunc='sum')` gives every type's total, but a type missing from the data (a month with no cancellations) raises `KeyError`. The `.where()` version never does.

Practice

- [ ] Add `Net = Deposit - Withdraw` and list the 10 clients with the highest net deposits.

3. np.where with AND, OR and NOT

`np.where(condition, value_if_true, value_if_false)` labels every row at once. Combine conditions with `&` (and), `|` (or), `~` (not), and put **each condition in its own brackets**.

```
one_pivot_v1['Traded Zero'] = np.where(
    (one_pivot_v1['Balance'] == 0) & (one_pivot_v1['Last Trade'] != 'No
Trade'),
    'Yes', 'No')

np.where(df['Type'].isin(['DEPOSIT', 'BONUS']), 'Money in', 'Other')        #
OR on one column
np.where(~df['Status'].isin(['demo', 'suspended']), 'Real', 'Excluded')     #
NOT
np.where((df['Status'] == 'active') & ((df['Type'] == 'DEPOSIT') |
(df['Type'] == 'BONUS')), 'Yes', 'No')
```

The bracket mistake

```
np.where((df['Balance'] == 0) & (df['Last Trade'] != 'No Trade', 'Yes',
'No'))   # wrong
```

The second condition's `)` closed after `'No'`, so Python built a tuple `(condition, 'Yes', 'No')` and `np.where` got one argument. Check that the brackets of each condition close before the first comma.

Errors you'll meet

You wrote	What happens	Fix
`cond1 and cond2`	`ValueError: The truth value of a Series is ambiguous`	`&` (and `|` for or)
`df['a'] > 0 & df['b'] > 0`	Wrong result or `TypeError`: `&` runs before `>`	`(df['a'] > 0) & (df['b'] > 0)`
`df['Type'] == 'DEPOSIT' or 'BONUS'`	True for every row	`df['Type'].isin(['DEPOSIT', 'BONUS'])`

Watch the data too

- A balance like `0.004` isn't `== 0`; use `.round(2).eq(0)` if it should count.
- If "no trade" is a blank date (NaT) rather than the text `'No Trade'`, use `.notna()` instead of `!= 'No Trade'`.
- Three or more outcomes: use `np.select`, not nested `np.where`.

4. Campaign flag: first deposit of 1,000 on 25 Aug

For any "first X" rule, pick the first row per client before testing it. Testing every deposit instead flags clients whose first deposit was something else and who only *also* deposited 1,000 that day, which was the first, wrong version.

```
dep = req_led[lt.eq('DEPOSIT')]

first_dep = (dep.sort_values('CreatedAt')                          # oldest
first
                .drop_duplicates(subset='ClientID', keep='first'))  # each
client's first deposit

camp_ids = set(first_dep.loc[(first_dep['CreatedAt'].dt.normalize() ==
pd.Timestamp('2026-08-25'))
                             & (first_dep['Credit'] == 1000), 'ClientID'])

one_pivot_v1['Campaign'] = np.where(one_pivot_v1['ClientID'].isin(camp_ids),
'Campaign', 'Not Campaign')
```

How it works

- **`sort_values('CreatedAt')`** + **`keep='first'`** leaves each client's earliest deposit. `ascending=False` would give the latest.
- **`.dt.normalize() == pd.Timestamp('2026-08-25')`** compares the date only, so any time that day matches.
- **`isin(camp_ids)`** with `np.where` turns the set of IDs into labels.
- **Check it:** merge `first_dep`'s date and amount next to the flag; every `Campaign` row should show 25 Aug and 1,000.

Three readings of one request

Request	Logic
Deposited 1,000 on 25 Aug	Test all deposits
First deposit was 1,000 on 25 Aug	Keep the first deposit, then test
Only deposit ever was 1,000 on 25 Aug	Also require one deposit in total

Two assumptions to confirm: "first" means first in the loaded ledger (check `led['CreatedAt'].min()`), and the date is in UTC unless converted to Dubai time first.

Practice

- [ ] Flag clients whose **last** deposit was in October and was over 5,000.

5. Trading days and their spread

Count distinct BILL dates per client for `Trading Days`, then join those dates into text for two "distribution" columns: every date, and days per month.

```
bills = (req_led[lt.eq('BILL')]
           .assign(Trade_Date=lambda d: d['CreatedAt'].dt.normalize())
           .drop_duplicates(['ClientID', 'Trade_Date'])           # one row
per client per day
           .sort_values(['ClientID', 'Trade_Date']))

trading_days = bills.groupby('ClientID', as_index=False)
['Trade_Date'].nunique() \
                    .rename(columns={'Trade_Date': 'Trading Days'})

trade_dates = (bills.groupby('ClientID')['Trade_Date']
                    .agg(lambda s: ', '.join(s.dt.strftime('%d %b')))
                    .rename('Trading Dates').reset_index())

per_month = (bills.assign(Month=lambda d: d['Trade_Date'].dt.to_period('M'))
                  .groupby(['ClientID',
'Month']).size().reset_index(name='days')
                  .assign(txt=lambda d: d['Month'].dt.strftime('%b %Y') + ':
' + d['days'].astype(str))
                  .groupby('ClientID')['txt'].agg(', '.join)
                  .rename('Days per Month').reset_index())
```

Merge each onto the client table with `how='left'`. Then `fillna(0).astype(int)` for `Trading Days`, and `fillna('No Trade')` for the two text columns.

How it works

- **`.dt.normalize()` + `drop_duplicates`** keeps one row per client per day, so 10 bills on one day count once.
- **Sorting before joining** makes the date list read oldest to newest.
- **`', '.join`** is pandas' `STRING_AGG`. Joining text per group has no built-in name, so a lambda belongs here.
- **`to_period('M')`** keeps months in calendar order. Sorting the text `'Aug 2026'` would put April first.
- **Check:** `Days per Month` adds up to `Trading Days`, and `Trading Days == 0` exactly where `Last Trade == 'No Trade'` (a `crosstab` of the two shows it).

Practice

- [ ] Add `Trading Weekdays`, e.g. `Mon: 4, Tue: 6, ...`, using `.dt.day_name().str[:3]`.

6. Excel export in the MIS design

Let pandas write the data starting on row 2, then style the same sheet with openpyxl before the file closes. Row 1 gets the coloured group bands, row 2 the gold headers, and the data gets borders and number formats.

```
from openpyxl.styles import Alignment, Border, Font, PatternFill, Side

with pd.ExcelWriter(path, engine='openpyxl') as writer:
    df.to_excel(writer, sheet_name='Broker Commission', index=False,
startrow=1)   # headers on row 2
    ws = writer.sheets['Broker Commission']

    ws.merge_cells(start_row=1, start_column=1, end_row=1, end_column=4)
# one band
    ws.cell(1, 1).value = 'Admin Panel Broker File'
    for c in range(1, 5):
        ws.cell(1, c).fill = PatternFill('solid', fgColor='1F3864')
    ws.cell(1, 1).font = Font(name='Arial', bold=True, size=12,
color='FFFFFF')

    for cell in ws[2]:
# headers
        cell.fill = PatternFill('solid', fgColor='FFE699')
        cell.font = Font(name='Arial', bold=True)

    for row in ws.iter_rows(min_row=3):
# money columns
        for cell in row[4:]:
            cell.number_format = '#,##0.00;[Red]-#,##0.00;"-"'

    ws.freeze_panes = 'B3'
    ws.auto_filter.ref = f"A2:I{ws.max_row}"
```

How it works

- **`startrow=1`** leaves row 1 empty for the bands (rows count from 0 in pandas, from 1 in Excel).
- **`writer.sheets[name]`** is the openpyxl sheet pandas just wrote, so styling happens in the same `with` block.
- **`merge_cells`** joins the band cells; fill every cell in the range so the colour is solid.
- **The number format has three parts:** positive; negative (red); zero (shown as `-`).
- **`freeze_panes = 'B3'`** freezes rows 1 to 2 and column A. **`auto_filter.ref`** adds filter buttons to the header row.

The full, reusable version is `broker_report_export.py`. It adds broker shading, a TOTAL row with live `SUM` formulas, and print setup.

Practice

- [ ] Add a TOTAL row under the data with `=SUM(E3:E{last})` for each money column, in bold.

7. Broker summary and the two-sheet workbook

Group the client rows by broker for totals and a list of children, apply the MIS rules, then write Brokers and Clients as two sheets styled by one shared function.

```
brokers = (result.groupby('Broker Client ID', as_index=False)
                 .agg(**{'Broker Name': ('Broker Name', 'first'),
                         'Children': ('Referred User Client ID', lambda s:
','.join(s.astype(str))),
                         'No of Children': ('Referred User Client ID',
'nunique'),
                         'Total Profit(Credit)': ('Profit', 'sum'),
                         'Total Loss(Debit)': ('Loss', 'sum'),
                         'Total Negative Balance Protection': ('NBP',
'sum')})
                 .rename(columns={'Broker Client ID': 'Parent'}))

brokers['Difference'] = brokers['Total Profit(Credit)'] - brokers['Total
Loss(Debit)']
is_loss = brokers['Difference'] < 0
added_nbp = brokers['Difference'] + brokers['Total Negative Balance
Protection']
```

```
brokers['Loss Status'] = np.where(is_loss, 'LOSS', 'NO LOSS')
brokers['Added NBP'] = added_nbp.where(is_loss, 'NOT ELIGIBLE FOR BROKERAGE')
brokers['Brokerage'] = ((-added_nbp).clip(lower=0) * 0.05).where(is_loss, 'NO
BROKERAGE')

with pd.ExcelWriter(path, engine='openpyxl') as writer:
    brokers.to_excel(writer, sheet_name='Brokers', index=False, startrow=1)
    result.to_excel(writer, sheet_name='Clients', index=False, startrow=1)
    style_sheet(writer.sheets['Brokers'], broker_bands, widths=[...],
money_cols=[5, 6, 7, 8, 10, 11])
    style_sheet(writer.sheets['Clients'], client_bands, widths=[...],
money_cols=[5, 6, 7, 8, 9])
```

How it works

- **`.agg(**{'Name': (column, function)})`** allows output names with spaces and brackets. Plain `name=(...)` only works for simple names.
- **`.where(is_loss, 'NOT ELIGIBLE...')`, not `np.where`:** with a number on one side and text on the other, `np.where` turns every number into text. `.where` keeps the numbers as numbers.
- **`(-added_nbp).clip(lower=0)`:** if NBP covers the whole loss, `Added NBP` turns positive and brokerage is 0, not 5% of a gain.
- **One `style_sheet()` function for both sheets** means one place to change the design.

The NBP filter bug

```
led_oct_bill = led_oct[led_oct['LedgerType'] == 'BILL']            # only
BILL rows left
led_child_fil = led_child[led_child['LedgerType'].isin(['BILL', 'NEGATIVE
BALANCE PROTECTION'])]  # NBP already gone
```

The NBP rows were removed one step before the filter meant to keep them, so NBP was always 0. Filter `led_oct` by both types instead.

Broker brokerage is not the sum of client commissions

`Brokerage` is 5% of the broker's **combined** loss, as in the MIS file. The Clients sheet's `Commission` is per client, where one client's profit can't offset another's loss. The two totals usually differ, so the payout rule has to say which one counts.

8. Importing your own modules from another folder

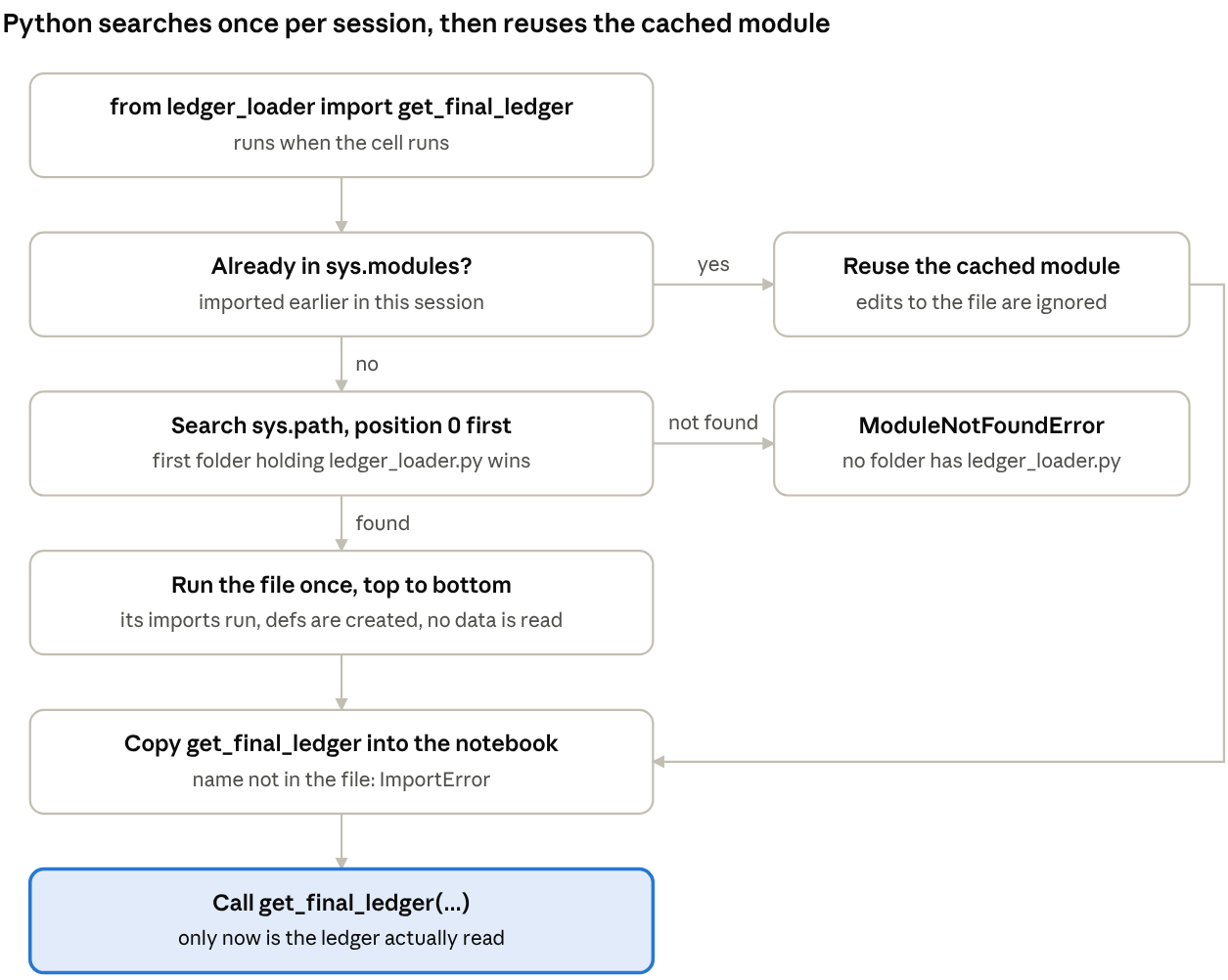
Python only finds modules in the folders listed in `sys.path`. Add the folder that holds `ledger_loader.py` to the front of that list, and the import works from any notebook.

```
import sys
sys.path.insert(0, r"C:\Users\Administrator\Videos\Office\migration")   # the
FOLDER, not the file

from ledger_loader import get_final_ledger
```

What the 0 means

`sys.path` is an ordinary list, and `insert(position, item)` needs both. `0` puts the folder **first**, so it's searched before every other folder and your file wins if another folder has a file with the same name. Leaving the 0 out raises `TypeError: insert expected 2 arguments, got 1`. `sys.path.append(folder)` takes one argument but searches the folder **last**.



what one import line does · 6 steps

The search happens once per kernel session. After that, Python reuses the cached copy, which is why edits need a restart or `%autoreload`.

Three ways, from quickest to most permanent

Way	How	Lasts
Per notebook	`sys.path.insert(0, r"C:\...\migration")`	Until the kernel restarts
Every notebook	`setx PYTHONPATH "C:\...\migration"` in a terminal, then restart VS Code	Permanently
One exact file	`importlib.util.spec_from_file_location(...)`	Per notebook

When something goes wrong

You see	Cause
`ModuleNotFoundError: No module named 'ledger_loader'`	Wrong folder, a typo, or the `sys.path` line wasn't run after a restart
`ImportError: cannot import name 'get_final_ledger'`	Different function name, or an older copy found first
Edits to the file have no effect	The module is cached in `sys.modules`; restart, or `%load_ext autoreload` then `%autoreload 2`

`print(ledger_loader.__file__)` shows which file was actually imported.

9. Writing functions: three levels

A function gives a block of work a name: inputs go in, one result comes out, and the steps are written once. The P&L chain was turned into a function at three levels. All three were tested and give identical results for October.

Level 1: wrap working code

```
def client_pnl_v1(led):
    summary = (led.assign(Loss=lambda x:
x['Debit'].where(x['LedgerType'].eq('BILL'), 0), ...)
                  .groupby('ClientID', as_index=False)[['Loss', 'Profit',
'NBP']].sum()
                  .assign(Net_Profit=..., Commission=...))
    return summary

pnl_v1 = client_pnl_v1(led_child_fil)
```

- **`def name(parameter):`** then an indented body. Defining runs nothing; **calling** `client_pnl_v1(...)` does.
- **The parameter `led`** is the name the input has *inside*. Your `led_child_fil` arrives as `led`.
- **`return`** sends the result out; without it you get `None`.

- **Names inside stay inside:** `summary` in the function can't touch anything in the notebook. Only `pnl_v1 = ...` outside creates or replaces a notebook variable.

Level 2: parameters and a default

```
def client_pnl_v2(led, start, end, commission_rate=0.05):
    """Loss, profit, NBP and commission per client, for start <= CreatedAt <
end."""
    rows = led[(led['CreatedAt'] >= start) & (led['CreatedAt'] < end)]
    ...
    return summary

client_pnl_v2(led, '2026-10-01', '2026-11-01')                          #
positional
client_pnl_v2(led, '2026-09-01', '2026-10-01', commission_rate=0.04)    #
keyword
```

- **Fixed values become parameters:** dates and the rate, so one function covers every month.
- **`commission_rate=0.05`** is a default: used unless you pass another value. Defaults go after parameters without one.
- **A docstring** (`"""..."""`) describes it; Shift+Tab after `client_pnl_v2(` shows it.

Level 3: optional filters, a check, and cleaning

```
def client_pnl_v3(led, start=None, end=None, clients=None,
commission_rate=0.05):
    needed = {'ClientID', 'CreatedAt', 'LedgerType', 'Debit', 'Credit'}
    missing = needed - set(led.columns)
    if missing:
        raise ValueError(f"The ledger is missing these columns:
{sorted(missing)}")

    rows = led
    if start is not None:
        rows = rows[rows['CreatedAt'] >= start]
    if clients is not None:
        rows = rows[rows['ClientID'].isin(clients)]
    ...
```

- **`None` means "not given"**, and each `if ... is not None` applies a filter only when asked.
- **`raise ValueError(...)`** stops early with your own message instead of a confusing pandas error halfway through.
- **`rows = led`, then `rows = rows[...]`** narrows step by step and never changes `led`.

Checking a function against a result you trust

```
pd.testing.assert_frame_equal(pnl.reset_index(drop=True),
pnl_v1.reset_index(drop=True), check_dtype=False)
```

It's silent when the tables match and raises an error showing the first difference when they don't. That's how you know a rewrite didn't change the numbers.

Rules of thumb

- Start from code that already works; wrap it and check the result is unchanged.
- Turn fixed values into parameters one at a time.
- One function, one job: calculating and exporting are separate functions.
- Return results; don't print them or change the input.

Problems and fixes

Two real errors, two silent wrong results, and one request read too broadly.

Problem	Tool	Cause	Fix	Section
`np.where` gets one argument	pandas	A `)` closed after `'No'`, making a tuple	Close each condition's bracket before the first comma	3
`Balance` may come from a random row	pandas	`sort_values('ClientID')` before `keep='first'`	`sort_values('CreatedAt', ascending=False)`	Review
Clients flagged whose first deposit wasn't the campaign one	pandas	Tested every deposit, not the first	Keep the first deposit per client, then test	4
NBP always 0 in the broker sheet	pandas	`led_oct_bill` dropped NBP rows before the NBP filter	Filter `led_oct` by both types	7

Problem	Tool	Cause	Fix	Section
`np.where` would turn Added NBP numbers into text	pandas	One side text, one side numbers	`Series.where(cond, 'text')`	7
`TypeError: insert expected 2 arguments, got 1`	Python	`sys.path.insert(folder)` without a position	`insert(0, folder)` or `append(folder)`	8
`KeyError` risk in the ledger-type pivot	pandas	A type missing in some months	`.where()` columns, or a `get` helper	2
Extra unnamed column in Excel	pandas	`to_excel` without `index=False`	`index=False`	—

Cheat sheet: new items

Only items not in earlier cheat sheets.

pandas and NumPy

Item	What it does	Example
`s.where(cond, 0)` then `groupby().sum()`	Conditional sum per group, no lambda	Deposits per client
`(a) & (b)`, `(a) | (b)`, `~a`	AND, OR, NOT on whole columns	`np.where` flags
`Series.where(cond, 'text')`	Keep numbers where true, text elsewhere, numbers stay numbers	`Added NBP`
`.agg(**{'Name with spaces': (col, func)})`	Output names that aren't valid Python names	Broker summary
`agg(', '.join)`	Join text per group	Children, trading dates
`.dt.to_period('M').dt.strftime('%b %Y')`	Month label that still sorts by date	`Oct 2026: 8`

Item	What it does	Example
`pd.testing.assert_frame_equal(a, b, check_dtype=False)`	Prove two tables are identical	Function vs original

Python

Item	What it does
`def f(a, b=5):` / `return x`	Define a function with a default; send a result out
`"""docstring"""`	Describe the function; Shift+Tab shows it
`if x is not None:`	Apply an optional input only when given
`raise ValueError(f"...")`	Stop with a clear message
`sys.path.insert(0, folder)`	Search this folder first when importing
`setx PYTHONPATH "folder"`	Make a folder importable everywhere (restart the editor)
`module.__file__`	Which file was imported
`%load_ext autoreload` + `%autoreload 2`	Pick up edits to imported files without restarting

openpyxl

Item	What it does
`df.to_excel(writer, startrow=1)` + `writer.sheets[name]`	Leave row 1 free, then style the written sheet
`ws.merge_cells(start_row=, start_column=, end_row=, end_column=)`	One title over several columns
`PatternFill('solid', fgColor='1F3864')`	Cell background colour
`'#,##0.00;[Red]-#,##0.00;"-"'`	Thousands, red negatives, dash for zero
`ws.freeze_panes = 'B3'`, `ws.auto_filter.ref = 'A2:I100'`	Frozen headers and filter buttons

Open items

Two fixes sit in code you've already run, and the payout rule needs a decision.

- [ ] Change the `balance` line to `sort_values('CreatedAt', ascending=False)`, then rerun `Balance` and `Traded Zero`.
- [ ] Filter `led_oct` (not `led_oct_bill`) by BILL and NBP, then rerun the broker workbook.
- [ ] Decide which brokerage rule the payout follows: the broker's combined loss, or the sum of client commissions.
- [ ] Confirm "first deposit" against the ledger's earliest date (`led['CreatedAt'].min()`), and whether 25 Aug is in UTC or Dubai time.
- [ ] Set `PYTHONPATH` once so `ledger_loader` and `nexora_metrics` import everywhere.
- [ ] Write `trading_days(led, start, end)` at Level 2 and check it with `assert_frame_equal`.
- [ ] Still open from earlier: the team-list checks, the Uno API endpoint and token, the first full October dialer run, and the mastery workbook's first `progress()`.